Reescrita de Consultas SQL Baseada em Modelos de Linguagem de Grande Escala (LLM-based SQL Query Rewriting)

Luiz Eduardo Treméa[1], **Fahad Kalil**[1]

[1]Ciência da Computação – Atitus Educação
Passo Fundo – RS – Brasil

1123471@atitus.edu.br, fahad.kalil@atitus.edu.br

Abstract. *Escreva seu resumo em língua estrangeira (inglês)...*

Resumo. *Resumo do trabalho (português)...*

1. Introdução

Bancos de dados relacionais estão por trás de boa parte dos sistemas que usamos todos os dias, de aplicações corporativas a plataformas de análise. À medida que esses sistemas crescem e as consultas ficam mais complexas, o tempo que cada consulta leva para ser executada deixa de ser um detalhe e passa a ter impacto direto no custo de infraestrutura e na experiência de quem depende daquela informação. Por isso, otimizar consultas SQL é um problema antigo e, ao mesmo tempo, sempre atual.

Tradicionalmente, esse trabalho fica a cargo do otimizador do próprio Sistema Gerenciador de Banco de Dados (SGBD), que escolhe um plano de execução com base em estimativas de custo e de cardinalidade. Ocorre que essas estimativas nem sempre são precisas, sobretudo quando há correlações complexas entre os dados ou quando a consulta foi escrita de forma pouco eficiente. Nesses casos, reescrever uma consulta mantendo o mesmo resultado, mas mudando como ela é expressa costuma render ganhos que o otimizador sozinho não alcança.

Os Modelos de Linguagem de Grande Escala (LLMs) tornam-se relevantes justamente nesse ponto. Trabalhos recentes mostram que esses modelos conseguem sugerir reescritas de consultas com bons resultados (Li; Yuan *et al.*, 2024; Sun *et al.*, 2025), mas também trazem um problema conhecido: a geração livre pode produzir consultas que parecem corretas e não são, seja por alucinação, seja por desvio semântico. Em um ambiente de produção, uma reescrita que retorna dados diferentes do esperado representa um risco inaceitável.

A justificativa deste trabalho parte justamente dessa tensão. Um LLM pode ampliar o espaço de otimização para além das regras fixas, mas só é útil de verdade se houver alguma forma de garantir que a consulta reescrita continua equivalente à original e que ela é, de fato, mais rápida. Em vez de confiar incondicionalmente no modelo, a proposta é deixá-lo sugerir e utilizar o próprio banco de dados como instância de validação.

Esse cenário conduz ao problema de pesquisa que orienta o estudo: é possível reescrever consultas SQL com o auxílio de LLMs de modo a melhorar o desempenho sem comprometer a corretude dos resultados?

Para responder a essa pergunta, define-se como objetivo geral desenvolver e avaliar um mecanismo de reescrita de consultas SQL baseado em LLMs, apoiado por etapas de validação sintática, semântica e de desempenho conduzidas pelo próprio SGBD.

Como objetivos específicos, o trabalho se propõe a:

- levantar, na literatura, as principais técnicas de otimização e de reescrita de consultas baseadas em aprendizado de máquina e em modelos de linguagem;
- projetar uma arquitetura não invasiva, capaz de intermediar a comunicação entre a consulta original, o LLM e o banco de dados;
- implementar um processo de validação que combine verificação sintática, comparação semântica de resultados e análise dos planos de execução;
- comparar o desempenho das consultas em três cenários sem reescrita, com reescrita sem validação e com reescrita validada;
- avaliar a viabilidade da abordagem para uso em ambientes reais, considerando os ganhos de desempenho e a preservação da semântica.

2. Referencial Teórico

O referencial que segue está organizado em cinco frentes. Começa pela evolução da otimização de consultas e pela entrada do aprendizado de máquina nesse processo; passa pela reescrita de consultas com modelos de linguagem, pela estimativa de cardinalidade e pelas arquiteturas que unem recuperação de conhecimento e agentes; e encerra com uma síntese crítica, que reúne o que a literatura já consolidou e expõe os limites teóricos, em especial o da equivalência de consultas, que recaem sobre a validação proposta neste trabalho.

2.1. Evolução da Otimização de Consultas e Uso de Inteligência Artificial

Durante décadas, a otimização de consultas em bancos de dados se apoiou sobre três pilares: a estimação de cardinalidade, o modelo de custo e a enumeração de planos. A qualidade do plano escolhido depende, em boa parte, de quão bem o otimizador prevê o volume de dados intermediários e de quão coerente é a função de custo em relação aos recursos disponíveis na máquina.

Esse modelo não é novo. Ainda no fim da década de 1970, o otimizador do System R formalizou a otimização baseada em custo: estimava a seletividade dos predicados e escolhia, por programação dinâmica, a ordem de junção mais barata entre as muitas possíveis (Selinger *et al.*, 1979). Décadas depois, o otimizador continua sendo descrito pelos mesmos três blocos: um conjunto de transformações que preservam o resultado, um modelo que estima o custo de cada alternativa e um algoritmo de busca que percorre o espaço de planos (Chaudhuri, 1998). Essa divisão importa para o presente trabalho: o LLM passa a sugerir as transformações, enquanto o SGBD segue responsável por estimar o custo e decidir.

Esse arranjo atende a muitos casos, mas revela limitações importantes em cenários mais exigentes, sobretudo quando existem correlações complexas entre atributos ou quando a carga de trabalho varia com frequência. Nessas situações, um erro de estimação pode levar o otimizador a escolher um plano físico inadequado, com perda direta de

desempenho (Marcus; Negi; Mao; Zhang *et al.*, 2019; Marcus; Negi; Mao; Tatbul *et al.*, 2021).

Foi diante desse tipo de limitação que se consolidou o paradigma de *Learned Query Optimization* (LQO), que traz o aprendizado de máquina para dentro do próprio processo de otimização. O *Neo*, por exemplo, emprega redes neurais profundas para aprender padrões de execução a partir de consultas reais e se adaptar ao comportamento dos dados (Marcus; Negi; Mao; Zhang *et al.*, 2019). O *Bao*, por sua vez, adota uma abordagem híbrida baseada em aprendizado por reforço, oferecendo sugestões de execução consulta a consulta e melhorando o desempenho em ambientes reais (Marcus; Negi; Mao; Tatbul *et al.*, 2021).

2.2. Reescrita de Consultas SQL com Modelos de Linguagem

Reescrever uma consulta SQL significa transformá-la em outra que produza exatamente o mesmo resultado, porém de forma mais eficiente. Diferentemente do que ocorre em abordagens de *Text-to-SQL*, aqui o ponto de partida já é uma consulta estruturada; o desafio está em alterá-la sem quebrar a equivalência lógica entre o que entra e o que sai.

A reescrita, aliás, não nasceu com os modelos de linguagem: ela já era um recurso conhecido dos próprios SGBDs. O Starburst, por exemplo, introduziu um otimizador extensível em que regras de reescrita eram aplicadas sobre a consulta antes do planejamento físico (Pirahesh; Hellerstein; Hasan, 1992), e essa tradição segue viva em frameworks como o Apache Calcite, que reúne uma ampla biblioteca de regras de transformação testadas e reaproveitáveis (Begoli *et al.*, 2018). O limite dessas abordagens é estrutural: elas só aplicam transformações que alguém previu e codificou de antemão. Os modelos de linguagem prometem ultrapassar esse teto, ao propor reescritas que não estão presas a um catálogo fixo de regras.

O *LLM-R2* recorre a modelos de linguagem para sugerir regras de reescrita e usa aprendizado contrastivo para refinar a forma como as consultas são representadas (Li; Yuan *et al.*, 2024). O *R-Bot* segue uma linha parecida, mas apoia o modelo na recuperação de evidências estruturais e semânticas, reduzindo os erros ao longo do processo (Sun *et al.*, 2025).

Há ainda propostas que vão além das regras fixas. O *QUITE* organiza múltiplos agentes em um fluxo controlado e testa estratégias de reescrita a partir do retorno do próprio banco de dados (Song *et al.*, 2025). Já o *LITHE* atua diretamente no espaço da consulta (*SQL-to-SQL*): orienta a busca por reescritas a partir das probabilidades de *tokens* do modelo e confirma a equivalência por meio de verificações de correção semântica fundamentadas em análise lógica e estatística (Dharwada *et al.*, 2026). Em uma linha próxima, o *GenRewrite* trata a reescrita de forma mais holística, indo além das regras fixas e expressando em linguagem natural as transformações que aprende (Liu; Mozafari, 2024).

Essa tarefa não deve ser confundida com um problema vizinho e mais conhecido, o *Text-to-SQL*, em que o modelo traduz uma pergunta em linguagem natural para uma consulta. Benchmarks como o *Spider* (Yu *et al.*, 2018) e o *BIRD* (Li; Hui *et al.*, 2023) mostram que mesmo modelos avançados ainda erram com frequência nessa tradução: no *BIRD*, a acurácia de execução de modelos do estado da arte ficou bem abaixo do desempenho humano. Esse dado é relevante por dois motivos: reforça que a saída de

um LLM não pode ser aceita sem verificação e evidencia que medir *eficiência*, e não apenas correção, é uma preocupação legítima ao avaliar SQL gerado por modelos. De forma mais ampla, levantamentos recentes situam a reescrita de consultas dentro de um movimento maior de aplicação de LLMs a tarefas de banco de dados (Li; Zhou; Zhao, 2024), movimento que esbarra sempre no mesmo ponto sensível: a alucinação, isto é, a tendência do modelo a produzir saídas plausíveis, porém incorretas (Huang *et al.*, 2024). Isso torna necessária uma instância externa que valide cada reescrita.

2.3. Estimativa de Cardinalidade e Modelos Aprendidos

A estimativa de cardinalidade costuma ser apontada como um dos fatores que mais pesam na qualidade do plano de execução, justamente por antecipar o volume de dados que será manipulado em cada etapa da consulta.

O *NeuroCard* ataca esse problema com redes neurais autorregressivas, capazes de capturar correlações complexas entre várias tabelas sem partir do pressuposto de que os atributos são independentes (Yang *et al.*, 2021). O *FLAT* segue outro caminho: combina fatoração independente e condicional em um modelo gráfico, em busca de um meio-termo entre precisão e custo computacional (Zhu *et al.*, 2021).

Boa parte dessa agenda foi impulsionada por uma constatação empírica. Ao avaliar otimizadores sobre o *Join Order Benchmark* (JOB), Leis et al. mostraram que os erros de estimativa de cardinalidade são o fator que mais degrada a qualidade dos planos, bem mais do que o modelo de custo ou o algoritmo de enumeração (Leis *et al.*, 2015). Esse achado explica tanto por que o JOB virou referência para avaliar estimadores quanto por que ele não é o foco deste trabalho, voltado à reescrita da consulta, e não à correção das estimativas.

Esses trabalhos apresentam um avanço claro frente aos métodos tradicionais, com estimativas mais confiáveis e reflexo direto no desempenho das consultas executadas.

2.4. Arquiteturas Baseadas em Recuperação e Agentes

Boa parte das propostas mais recentes combina modelos de linguagem com mecanismos de recuperação de conhecimento, formando pipelines no estilo *Retrieval-Augmented Generation* (RAG). A ideia é converter as consultas em representações vetoriais e, a partir delas, recuperar evidências que ajudem a orientar a geração.

Esses pipelines se apoiam em técnicas que se consolidaram no estudo dos próprios modelos de linguagem. O aprendizado em contexto, em que o modelo generaliza a partir de poucos exemplos fornecidos no próprio *prompt*, mostrou que essas redes resolvem tarefas novas sem treinamento adicional (Brown *et al.*, 2020), mecanismo que, no presente trabalho, corresponde a fornecer o esquema do banco e exemplos junto da consulta. A isso somam-se estratégias de raciocínio passo a passo (Wei *et al.*, 2022) e de autoconsistência, em que várias respostas são amostradas e a mais coerente prevalece (Wang *et al.*, 2023), ideia próxima, em espírito, da etapa de validação que descarta reescritas inválidas. A geração aumentada por recuperação (RAG), por fim, combina a memória interna do modelo com a busca de evidências externas, reduzindo respostas sem fundamento (Lewis *et al.*, 2020).

O *R-Bot* ilustra bem essa estratégia ao unir recuperação híbrida e geração guiada por evidências (Sun *et al.*, 2025). O *QUITE* chega a um resultado semelhante por

outro meio, distribuindo entre diferentes agentes as tarefas de planejamento, validação e execução das reescritas, em um processo iterativo (Song *et al.*, 2025).

Esses sistemas sugerem que apoiar o modelo em conhecimento externo e em validações sucessivas reduz erros e torna a solução mais robusta (Sun *et al.*, 2025; Song *et al.*, 2025; Dharwada *et al.*, 2026).

2.5. Síntese Crítica da Literatura

A literatura recente aponta para dois caminhos principais. O primeiro leva o aprendizado de máquina para dentro do otimizador, como fazem o *Neo*, o *Bao*, o *NeuroCard* e o *FLAT* (Marcus; Negi; Mao; Zhang *et al.*, 2019; Marcus; Negi; Mao; Tatbul *et al.*, 2021; Yang *et al.*, 2021; Zhu *et al.*, 2021).

O segundo mantém o otimizador intacto e atua de fora, reescrevendo as consultas com o apoio de modelos de linguagem, como é o caso do *LLM-R2*, do *R-Bot*, do *QUITE* e do *LITHE* (Li; Yuan *et al.*, 2024; Sun *et al.*, 2025; Song *et al.*, 2025; Dharwada *et al.*, 2026).

Há, porém, um limite teórico que nenhuma dessas linhas elimina e que recai diretamente sobre a validação proposta neste trabalho: decidir se duas consultas são equivalentes é, no caso geral, um problema indecidível, pois deriva da indecidibilidade da lógica de primeira ordem sobre instâncias finitas (Abiteboul; Hull; Vianu, 1995). Mesmo quando o problema se restringe às consultas conjuntivas, decidir contenção e equivalência continua sendo NP-completo (Chandra; Merlin, 1977). Por isso, as ferramentas que provam equivalência de forma automática cobrem apenas fragmentos do SQL: o *Cosette* decide a equivalência para uma classe de consultas, mas não trata recursos como valores nulos e diferença de conjuntos (Chu *et al.*, 2017); o *EQUITAS* traduz consultas para fórmulas lógicas e as submete a um resolvedor SMT (Zhou; Arulraj; Navathe; Harris; Xu, 2019); o *SPES* leva essa ideia para a semântica de multiconjunto, mais fiel ao SQL real (Zhou; Arulraj; Navathe; Harris; Wu, 2022); e o *VeriEQL*, embora suporte uma linguagem mais ampla, realiza verificação limitada, restrita a domínios finitos (He *et al.*, 2024).

Esse panorama tem uma consequência prática para a metodologia adotada aqui. Como não há como provar, de forma geral e eficiente, que a consulta reescrita é equivalente à original, a validação semântica deste trabalho recorre à comparação dos resultados produzidos sobre um conjunto concreto de dados. É uma aproximação pragmática, não uma garantia formal: resultados diferentes refutam a equivalência, mas resultados iguais sobre uma instância não a comprovam em definitivo. Essa limitação é assumida de forma explícita e está alinhada à prática dos próprios sistemas de reescrita por LLM, que também validam suas transformações por execução.

Independentemente do caminho, os desafios em aberto se repetem: garantir a correção semântica, controlar a latência dos modelos e integrar tudo isso a sistemas que já estão em produção. O encontro entre aprendizado de máquina, reescrita de consultas e recuperação de conhecimento parece concentrar o terreno mais promissor para os próximos avanços.

3. Trabalhos Relacionados

Esta seção reúne os principais estudos que aplicam Inteligência Artificial, em especial LLMs e modelos aprendidos, à reescrita e à otimização de consultas SQL. Ao final,

destaca-se em que a proposta deste trabalho se diferencia deles.

- **O uso da Inteligência Artificial na otimização de consultas em bancos de dados** (Amaral; Andrade; Campos, 2025)
  Os autores investigam até que ponto o ChatGPT é capaz de reescrever consultas de forma semântica em ambientes PostgreSQL. Para isso, montam um experimento com scripts em Python que automatizam a execução das consultas e a coleta de métricas, com foco em transformações como o uso de *Common Table Expressions* (CTEs), pré-agregações e redução de subconsultas. Em cenários complexos, com muitos *joins*, a reescrita assistida por IA chegou a reduzir o tempo de execução em até 90%, além de racionalizar o uso de índices e tabelas.
- **LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System** (Li; Yuan *et al.*, 2024)
  O LLM-R2 usa LLMs para sugerir sequências de regras de reescrita voltadas a plataformas como o Apache Calcite. Seu principal trunfo é um modelo contrastivo, treinado por currículo, que seleciona demonstrações (*few-shot*) estruturalmente parecidas com a consulta de entrada. Como a execução final das regras fica a cargo do próprio motor do banco, o sistema garante que a reescrita seja executável e semanticamente equivalente, algo que abordagens de geração livre, mais sujeitas a alucinações, nem sempre conseguem.
- **R-Bot: An LLM-based Query Rewrite System** (Sun *et al.*, 2025)
  O R-Bot busca reduzir alucinações apoiando-se em uma arquitetura de *Retrieval-Augmented Generation* (RAG). A recuperação é híbrida: combina características estruturais e semânticas para localizar "receitas de reescrita" e evidências em fóruns técnicos e manuais de SGBDs. A reescrita acontece passo a passo, com autorreflexão, o que permite ao modelo entender como as regras se relacionam antes de aplicá-las.
- **LITHE: A Query Rewrite Advisor using LLMs** (Dharwada *et al.*, 2026)
  O LITHE trabalha diretamente no espaço da consulta (*SQL-to-SQL*), e não no espaço do plano físico. Ele aplica regras sensíveis ao banco, apoiadas em seletividade e metadados, e as combina com uma busca em árvore de Monte Carlo (*Monte Carlo Tree Search*, MCTS) guiada pelas probabilidades de *tokens* do LLM. Quando a confiança do modelo é baixa, essa estratégia abre espaço para explorar vários caminhos de reescrita, alcançando ganhos expressivos em relação aos otimizadores nativos.
- **QUITE: A Query Rewrite System Beyond Rules with LLM Agents** (Song *et al.*, 2025)
  O QUITE é um *framework* de múltiplos agentes coordenados por uma Máquina de Estados Finita (*Finite State Machine*, FSM). Ele recebe *feedback* em tempo real do banco por meio de comandos `EXPLAIN` e injeta *hints* para garantir que o plano otimizado seja de fato seguido na execução. Por não depender apenas de regras fixas, consegue encontrar otimizações que transformações pré-definidas não alcançariam.

Diferentemente das abordagens de otimização interna, como o Neo e o Bao, que substituem ou modificam partes do otimizador do SGBD, e dos sistemas presos a regras

fixas ou voltados apenas a reduzir latência, este trabalho propõe uma reescrita *SQL-to-SQL* baseada em LLMs, com validação e análise conduzidas pelo próprio gerenciador de banco de dados.

A proposta se diferencia por buscar um equilíbrio entre desempenho e preservação semântica. A geração livre oferece mais liberdade de transformação, mas convive com *semantic drift*, alucinações e regressões de desempenho; os sistemas rígidos, por outro lado, ficam limitados às transformações que alguém previu e codificou de antemão. A ideia aqui é colocar o LLM como agente de reescrita semântica em uma camada externa e não invasiva, enquanto o SGBD assume o papel de validar e fornecer evidências técnicas.

A verificação acontece em etapas. Primeiro, a consulta reescrita passa pelo *parser* do banco, que confere sua validade sintática. Em seguida, comandos como `EXPLAIN` e `EXPLAIN ANALYZE` permitem analisar o plano de execução e comparar o custo estimado entre a versão original e a reescrita. Por fim, comparam-se os resultados das duas consultas para confirmar que a transformação não alterou a semântica original.

O ciclo completo funciona assim: o LLM propõe a transformação, o otimizador do SGBD aponta os ganhos de desempenho e os mecanismos de validação asseguram que a reescrita está correta. A meta é chegar a uma solução prática, explicável e compatível com diferentes SGBDs relacionais, confiável o bastante para uso corporativo e ambientes de produção.

4. Materiais e Métodos

Este trabalho se caracteriza como uma pesquisa aplicada, de abordagem quantitativa e natureza experimental. É aplicada porque parte de um problema concreto, a otimização de consultas SQL em bancos de dados; é quantitativa porque recorre a métricas objetivas de desempenho para avaliar os resultados; e é experimental porque envolve a construção de um protótipo funcional e a realização de testes controlados em ambiente computacional.

A proposta é desenvolver um mecanismo de reescrita de consultas SQL baseado em Modelos de Linguagem de Grande Escala (*Large Language Models*, LLMs), acompanhado de etapas de validação sintática e semântica e de uma análise de desempenho apoiada no *feedback* do próprio otimizador. Em essência, a pergunta que orienta o estudo é direta: consultas reescritas automaticamente conseguem rodar mais rápido sem comprometer a corretude dos resultados?

4.1. Arquitetura da Solução Proposta

A arquitetura proposta funciona como uma camada intermediária, encarregada de receber as consultas, encaminhá-las ao modelo de linguagem e avaliar se as transformações sugeridas fazem sentido.

O fluxo começa com uma consulta SQL original submetida ao sistema. Ela é então enviada ao LLM, que recebe instruções específicas para devolver uma versão semanticamente equivalente e, idealmente, mais eficiente. Gerada a nova consulta, entram em cena as etapas de validação, responsáveis por verificar tanto a corretude quanto o impacto sobre o desempenho.

A solução foi concebida para ser não invasiva: ela não exige nenhuma alteração no núcleo do Sistema Gerenciador de Banco de Dados (SGBD) e pode ser acoplada como

complemento aos otimizadores que já existem.

4.2. Processo de Reescrita de Consultas

O processo se inicia com a submissão de uma consulta SQL ao sistema. Essa consulta original serve de entrada para o modelo de linguagem, que recebe um conjunto de instruções voltadas a melhorar a estrutura lógica da consulta.

Entre as transformações esperadas estão a troca de subconsultas por junções, o uso de *Common Table Expressions* (CTEs), a simplificação de predicados, a reorganização de filtros e a eliminação de operações redundantes, estratégias que a literatura já associa à redução de custos de execução e ao melhor aproveitamento dos recursos computacionais.

O modelo não tem autonomia para substituir a consulta original por conta própria. Toda reescrita precisa passar pelas etapas de validação definidas neste trabalho antes de ser aceita como alternativa válida.

4.3. Validação das Consultas Reescritas

Como a literatura registra riscos reais de alucinação e de desvio semântico, a validação foi organizada em múltiplos estágios.

O primeiro é a validação sintática, feita pelo próprio *parser* do banco: consultas com erro de compilação ou incompatibilidades estruturais são descartadas de imediato.

Depois vem a validação semântica. Aqui, a consulta original e a reescrita rodam sobre o mesmo conjunto de dados e seus resultados são comparados; havendo qualquer diferença entre os conjuntos retornados, a reescrita é rejeitada.

Por último, a validação de desempenho se apoia nas informações do otimizador. Os planos gerados por `EXPLAIN` e `EXPLAIN ANALYZE` permitem comparar custos estimados, operadores e tempos de execução. Só é considerada uma otimização válida a consulta que preserva a equivalência semântica e, ao mesmo tempo, melhora o desempenho.

4.4. Ambiente Experimental

Os experimentos serão conduzidos no PostgreSQL, escolhido tanto por sua presença consolidada em ambientes acadêmicos e corporativos quanto pelas ferramentas nativas de análise de desempenho que oferece.

O protótipo será escrito em Python, linguagem que se integra com facilidade a bancos relacionais e a plataformas de Inteligência Artificial. Ainda, entre as bibliotecas Python necessárias, estão: SQLAlchemy, Psycopg, Pandas e Matplotlib.

O SQLAlchemy permite a definição e execução de consultas SQL, enquanto o Psycopg atuará como driver de comunicação com o PostgreSQL. Já o Pandas será usado para manipular e comparar os resultados retornados pelas consultas e, por fim, Matplotlib para gerar os gráficos e apoiar as análises estatísticas.

4.5. Modelo de Linguagem Utilizado

A reescrita propriamente dita ficará a cargo de um Modelo de Linguagem de Grande Escala capaz de lidar com tarefas de *SQL-to-SQL Query Rewriting*.

A depender da disponibilidade de acesso e dos recursos computacionais ao longo do estudo, os experimentos poderão recorrer a modelos como GPT-4o, GPT-5, DeepSeek ou Llama 3. A decisão levará em conta a qualidade das reescritas produzidas, a estabilidade dos resultados e a viabilidade de uso dentro do ambiente experimental montado.

4.6. Conjunto de Dados

A avaliação utilizará bases consagradas na literatura de otimização de consultas e desempenho de bancos de dados. Entre as consideradas estão o TPC-H (Transaction Processing Performance Council, 2022), o TPC-DS (Nambiar; Poess, 2006) e o *Join Order Benchmark* (JOB) (Leis *et al.*, 2015), montado a partir da base IMDb. Esses benchmarks reúnem consultas analíticas de diferentes níveis de complexidade, com múltiplas junções, agregações e operações de filtragem. Trabalhar com eles ajuda tanto a comparar os resultados com os de trabalhos relacionados quanto a garantir maior reprodutibilidade dos experimentos.

4.7. Cenários de Avaliação

A avaliação será feita com três cenários distintos. No primeiro, as consultas rodam sem nenhuma alteração, apenas com o otimizador nativo do PostgreSQL. No segundo, passam pela reescrita do modelo de linguagem antes de serem executadas, mas sem nenhum mecanismo adicional de validação. No terceiro, as consultas reescritas percorrem todo o processo de validação proposto aqui, com as verificações sintática, semântica e de plano de execução.

Comparar três cenários permite enxergar não só o potencial de otimização dos LLMs, mas também o quanto os mecanismos de validação ajudam a evitar regressões de desempenho e erros semânticos.

4.8. Métricas de Avaliação

A análise se baseará em métricas objetivas de desempenho: o tempo total de execução, o custo estimado do plano, a quantidade de linhas processadas, a taxa de sucesso das reescritas, o percentual de consultas efetivamente otimizadas e o número de reescritas rejeitadas ao longo da validação.

Esses números serão comparados estatisticamente entre os cenários, o que deve indicar se a abordagem proposta é eficaz e se faz sentido aplicá-la em ambientes reais.

Referências

ABITEBOUL, S.; HULL, R.; VIANU, V. **Foundations of Databases**. Reading, MA: Addison-Wesley, 1995.

AMARAL, K. P. N. d.; ANDRADE, L. P. d.; CAMPOS, E. A. V. d. O uso da Inteligência Artificial na otimização de consultas em bancos de dados. **Revista Camalotes – RECAM**, 2025. Disponível em: https://periodicos.insted.edu.br/recam/index. Acesso em: 9 jun. 2026.

BEGOLI, E. *et al.* Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. *In:* PROCEEDINGS of the 2018 International Conference on Management of Data (SIGMOD). [*S. l.*]: ACM, 2018. p. 221–230. DOI: 10.1145/3183713.3190662.

BROWN, T. B. *et al.* Language Models are Few-Shot Learners. *In:* ADVANCES in Neural Information Processing Systems 33 (NeurIPS). [*S. l.: s. n.*], 2020. p. 1877–1901. Disponível em: https://arxiv.org/abs/2005.14165. Acesso em: 16 jun. 2026.

CHANDRA, A. K.; MERLIN, P. M. Optimal Implementation of Conjunctive Queries in Relational Data Bases. *In:* PROCEEDINGS of the Ninth Annual ACM Symposium on Theory of Computing (STOC). [*S. l.*]: ACM, 1977. p. 77–90. DOI: 10.1145/800105.803397.

CHAUDHURI, S. An Overview of Query Optimization in Relational Systems. *In:* PROCEEDINGS of the Seventeenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS). [*S. l.*]: ACM, 1998. p. 34–43. DOI: 10.1145/275487.275492.

CHU, S. *et al.* Cosette: An Automated Prover for SQL. *In:* 8TH Biennial Conference on Innovative Data Systems Research (CIDR). [*S. l.: s. n.*], 2017. Disponível em: https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf. Acesso em: 16 jun. 2026.

DHARWADA, S. *et al.* LITHE: A Query Rewrite Advisor using LLMs. *In:* INTERNATIONAL CONFERENCE ON EXTENDING DATABASE TECHNOLOGY. **Proceedings of the 29th International Conference on Extending Database Technology (EDBT 2026)**. Tampere, Finland: OpenProceedings.org, 2026. Disponível em: https://openproceedings.org/2026/conf/edbt/paper-93.pdf. Acesso em: 9 jun. 2026.

HE, Y. *et al.* VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints. **Proceedings of the ACM on Programming Languages**, v. 8, OOPSLA1, p. 1–29, 2024. Disponível em: https://arxiv.org/abs/2403.03193. Acesso em: 16 jun. 2026.

HUANG, L. *et al.* A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. **ACM Transactions on Information Systems**, 2024. Disponível em: https://arxiv.org/abs/2311.05232. Acesso em: 16 jun. 2026.

LEIS, V. *et al.* How Good Are Query Optimizers, Really? **Proceedings of the VLDB Endowment**, v. 9, n. 3, p. 204–215, 2015.

LEWIS, P. *et al.* Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *In:* ADVANCES in Neural Information Processing Systems 33 (NeurIPS). [*S. l.: s. n.*], 2020. p. 9459–9474. Disponível em: https://arxiv.org/abs/2005.11401. Acesso em: 16 jun. 2026.

LI, G.; ZHOU, X.; ZHAO, X. LLM for Data Management. **Proceedings of the VLDB Endowment**, v. 17, n. 12, p. 4213–4216, 2024.

LI, J.; HUI, B. *et al.* Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. *In:* ADVANCES in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track. [*S. l.: s. n.*], 2023. Disponível em: https://arxiv.org/abs/2305.03111. Acesso em: 16 jun. 2026.

LI, Z.; YUAN, H. *et al.* LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. **Proceedings of the VLDB Endowment**, v. 18, n. 1, p. 53–65, 2024. Disponível em: https://www.vldb.org/pvldb/vol18/p53-yuan.pdf. Acesso em: 9 jun. 2026.

LIU, J.; MOZAFARI, B. **GenRewrite: Query Rewriting via Large Language Models**. [*S. l.: s. n.*], 2024. arXiv preprint, abs/2403.09060. DOI: 10.48550/arXiv.2403.09060. Disponível em: https://arxiv.org/abs/2403.09060. Acesso em: 16 jun. 2026.

MARCUS, R.; NEGI, P.; MAO, H.; TATBUL, N. *et al.* Bao: Making Learned Query Optimization Practical. *In:* ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA. **Proceedings of the 2021 International Conference on Management of Data**. Virtual Event, China: ACM, 2021. p. 1275–1288. DOI: 10.1145/3448016.3452838. Disponível em: https://people.csail.mit.edu/hongzi/content/publications/BAO-Sigmod21.pdf. Acesso em: 9 jun. 2026.

MARCUS, R.; NEGI, P.; MAO, H.; ZHANG, C. *et al.* Neo: A Learned Query Optimizer. **Proceedings of the VLDB Endowment**, v. 12, n. 11, p. 1705–1718, 2019. Disponível em: https://www.vldb.org/pvldb/vol12/p1705-marcus.pdf. Acesso em: 9 jun. 2026.

NAMBIAR, R. O.; POESS, M. The Making of TPC-DS. *In:* PROCEEDINGS of the 32nd International Conference on Very Large Data Bases (VLDB). [*S. l.*]: VLDB Endowment, 2006. p. 1049–1058.

PIRAHESH, H.; HELLERSTEIN, J. M.; HASAN, W. Extensible/Rule Based Query Rewrite Optimization in Starburst. *In:* PROCEEDINGS of the 1992 ACM SIGMOD International Conference on Management of Data. [*S. l.*]: ACM, 1992. p. 39–48. DOI: 10.1145/130283.130294.

SELINGER, P. G. *et al.* Access Path Selection in a Relational Database Management System. *In:* PROCEEDINGS of the 1979 ACM SIGMOD International Conference on Management of Data. [*S. l.*]: ACM, 1979. p. 23–34. DOI: 10.1145/582095.582099.

SONG, Y. *et al.* **QUITE: A Query Rewrite System Beyond Rules with LLM Agents**. [*S. l.: s. n.*], 2025. arXiv preprint, abs/2506.07675. DOI: 10.48550/arXiv.2506.07675. Disponível em: https://arxiv.org/abs/2506.07675. Acesso em: 9 jun. 2026.

SUN, Z. *et al.* R-Bot: An LLM-based Query Rewrite System. **Proceedings of the VLDB Endowment**, v. 18, n. 12, p. 5031–5044, 2025. Disponível em: https://www.vldb.org/pvldb/vol18/p5031-li.pdf. Acesso em: 9 jun. 2026.

TRANSACTION PROCESSING PERFORMANCE COUNCIL. **TPC Benchmark H (TPC-H) Standard Specification**. [*S. l.: s. n.*], 2022. Revision 3.0.1. Disponível em: https://www.tpc.org/tpch/. Acesso em: 16 jun. 2026.

WANG, X. *et al.* Self-Consistency Improves Chain of Thought Reasoning in Language Models. *In:* THE Eleventh International Conference on Learning Representations (ICLR). [*S. l.: s. n.*], 2023. Disponível em: https://arxiv.org/abs/2203.11171. Acesso em: 16 jun. 2026.

WEI, J. *et al.* Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *In:* ADVANCES in Neural Information Processing Systems 35 (NeurIPS). [*S. l.: s. n.*], 2022. p. 24824–24837. Disponível em: https://arxiv.org/abs/2201.11903. Acesso em: 16 jun. 2026.

YANG, Z. *et al.* NeuroCard: One Cardinality Estimator for All Tables. **Proceedings of the VLDB Endowment**, v. 14, n. 1, p. 61–73, 2021. Disponível em: https://www.vldb.org/pvldb/vol14/p61-yang.pdf. Acesso em: 9 jun. 2026.

YU, T. *et al.* Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *In:* PROCEEDINGS of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP). [*S. l.: s. n.*], 2018. p. 3911–3921. Disponível em: https://arxiv.org/abs/1809.08887. Acesso em: 16 jun. 2026.

ZHOU, Q.; ARULRAJ, J.; NAVATHE, S. B.; HARRIS, W.; WU, J. SPES: A Symbolic Approach to Proving Query Equivalence Under Bag Semantics. *In:* 2022 IEEE 38th International Conference on Data Engineering (ICDE). [*S. l.*]: IEEE, 2022. p. 2735–2748. DOI: 10.1109/ICDE53745.2022.00250.

ZHOU, Q.; ARULRAJ, J.; NAVATHE, S. B.; HARRIS, W.; XU, D. Automated Verification of Query Equivalence Using Satisfiability Modulo Theories. **Proceedings of the VLDB Endowment**, v. 12, n. 11, p. 1276–1288, 2019.

ZHU, R. *et al.* FLAT: Fast, Lightweight and Accurate Method for Cardinality Estimation. **Proceedings of the VLDB Endowment**, v. 14, n. 9, p. 1489–1502, 2021. Disponível em: https://doi.org/10.14778/3461535.3461539. Acesso em: 9 jun. 2026.